

Ejercicio 15: Formas de Parentizar una Expresión Lógica

Enunciado

Dada una expresión lógica con los símbolos `true` , `false` , `and` , `or` y `xor` , contar el número de formas de poner paréntesis de manera que la expresión se evalúe como `true` .

Ejemplo: Para `true and false xor true` , solo hay una forma de agruparla que resulte en `true` .

1. Modelado del Problema

Este problema se asemeja al de la *Multiplicación de una Cadena de Matrices*. La clave es que cualquier expresión de longitud n se puede dividir en dos subexpresiones mediante un operador principal en alguna posición k .

Definiciones

- Sea S la secuencia de símbolos (valores lógicos) y O la secuencia de operadores.
- Si tenemos n valores lógicos, tendremos $n - 1$ operadores.
- Debido a la naturaleza de los operadores (`AND` , `OR` , `XOR`), para saber cuántas formas hay de obtener `true` , necesitamos conocer también cuántas formas hay de obtener `false` .

2. Estructura de Programación Dinámica

Definición de las Funciones

Usaremos dos tablas de programación dinámica, $T[i][j]$ y $F[i][j]$:

- $T[i][j]$: Número de formas de parentizar la subexpresión desde el símbolo i hasta el j para que resulte **true**.
- $F[i][j]$: Número de formas de parentizar la subexpresión desde el símbolo i hasta el j para que resulte **false**.

Relación de Recurrencia

Para un intervalo $[i, j]$, iteramos sobre cada operador k situado entre i y $j - 1$. Sea $Total = (T[i][k] + F[i][k]) \times (T[k + 1][j] + F[k + 1][j])$.

Dependiendo del operador en la posición k :

1. Si el operador es `AND` :
 - $T[i][j] += T[i][k] \times T[k + 1][j]$
 - $F[i][j] += Total - (T[i][k] \times T[k + 1][j])$
2. Si el operador es `OR` :

- $T[i][j] += Total - (F[i][k] \times F[k+1][j])$
- $F[i][j] += F[i][k] \times F[k+1][j]$

3. Si el operador es XOR :

- $T[i][j] += (T[i][k] \times F[k+1][j]) + (F[i][k] \times T[k+1][j])$
- $F[i][j] += (T[i][k] \times T[k+1][j]) + (F[i][k] \times F[k+1][j])$

Casos Base

Para intervalos de longitud 1 (un solo símbolo i):

- Si $S[i] == \text{'true'}$: $T[i][i] = 1, F[i][i] = 0$.
- Si $S[i] == \text{'false'}$: $T[i][i] = 0, F[i][i] = 1$.

3. Algoritmo y Complejidad

Pseudocódigo

```

Entrada: simbolos[1..n], operadores[1..n-1]

Para i de 1 a n:
    Si simbolos[i] == 'true': T[i][i]=1, F[i][i]=0
    Sino: T[i][i]=0, F[i][i]=1

Para longitud de 2 a n:
    Para i de 1 a n - longitud + 1:
        j = i + longitud - 1
        Para k de i a j-1:
            // Aplicar las fórmulas de recurrencia según operadores[k]
            // sumando a T[i][j] y F[i][j]

Retornar T[1][n]
```

Análisis de Coste

- **Temporal:** $O(n^3)$. Tenemos dos bucles para definir el intervalo (n^2) y un tercer bucle interno para probar todos los posibles puntos de división k (n).
- **Espacial:** $O(n^2)$ para almacenar las dos tablas T y F .